TECHNISCHE UNIVERSITÄT BERLIN

Scalable Distributed URL Shortener

Maurice Kühl, Tom-Hendrik Lübke, Ngoc Han Nguyen | Scalability Engineering | 13th June 2026

Core Functionality

Create Short URL:

You submit a long URL and get a shortened URL back

➔ https://example.com/some/long/url → http://some.domain/a1B2c3D4

Access Short URL:

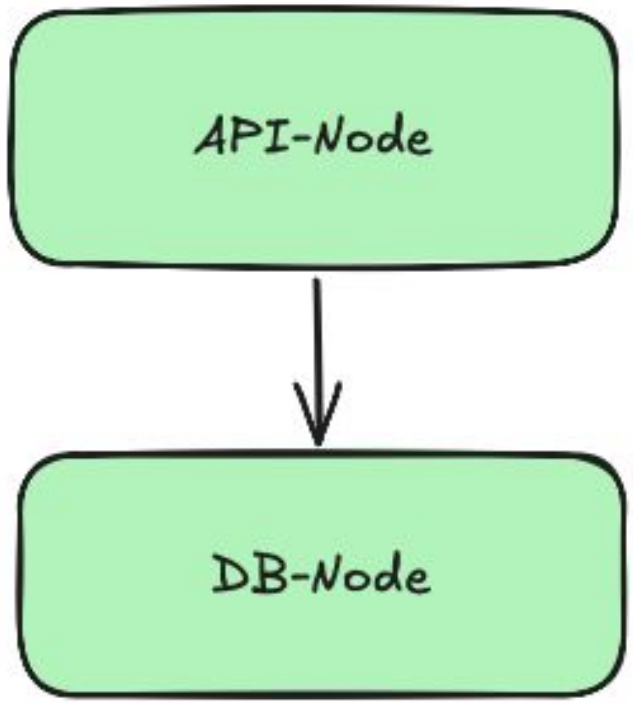
You request a shortened URL and receive a redirect to the original URL in response

➔ http://some.domain/a1B2c3D4 → https://example.com/some/long/url

[1]

The Starting Point

The Starting Point



The Starting Point - Results

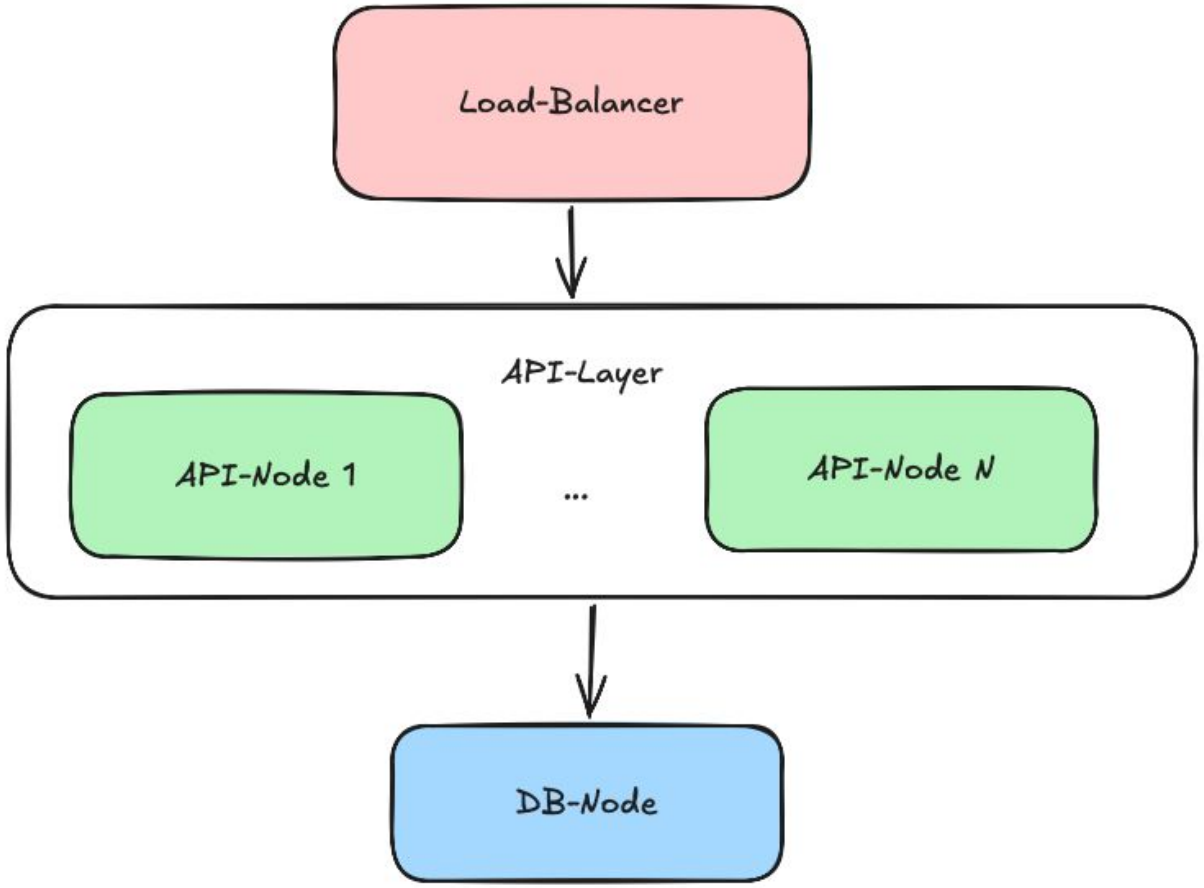
- Throughput ~300 req/s
- API-Node as bottleneck

The Starting Point - Results

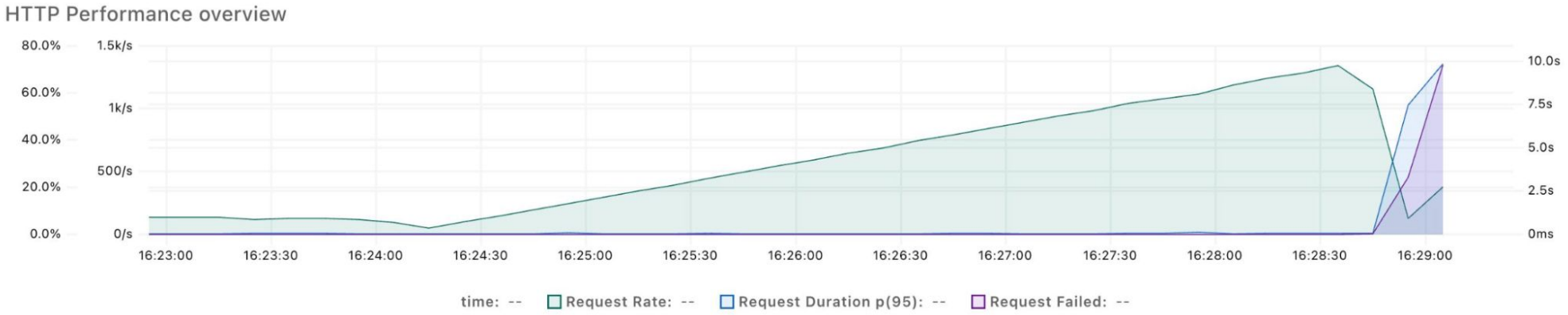
- Throughput ~300 req/s
- API-Node as bottleneck

➔ **Make the API-Node horizontally scalable**

Scalable API-Nodes

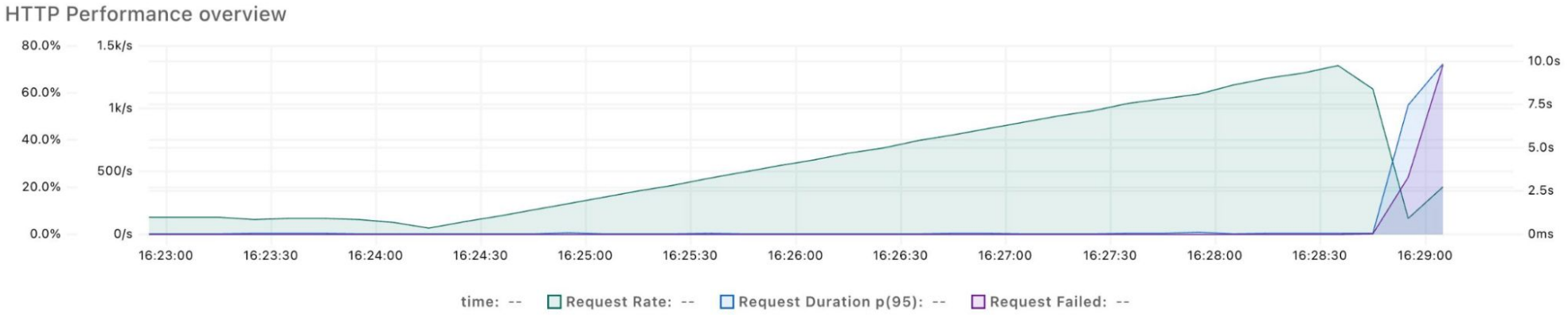


Scalable API-Nodes - Results



- 5 API-Nodes
- Throughput ~1,5k req/s
- DB-Node becomes the bottleneck

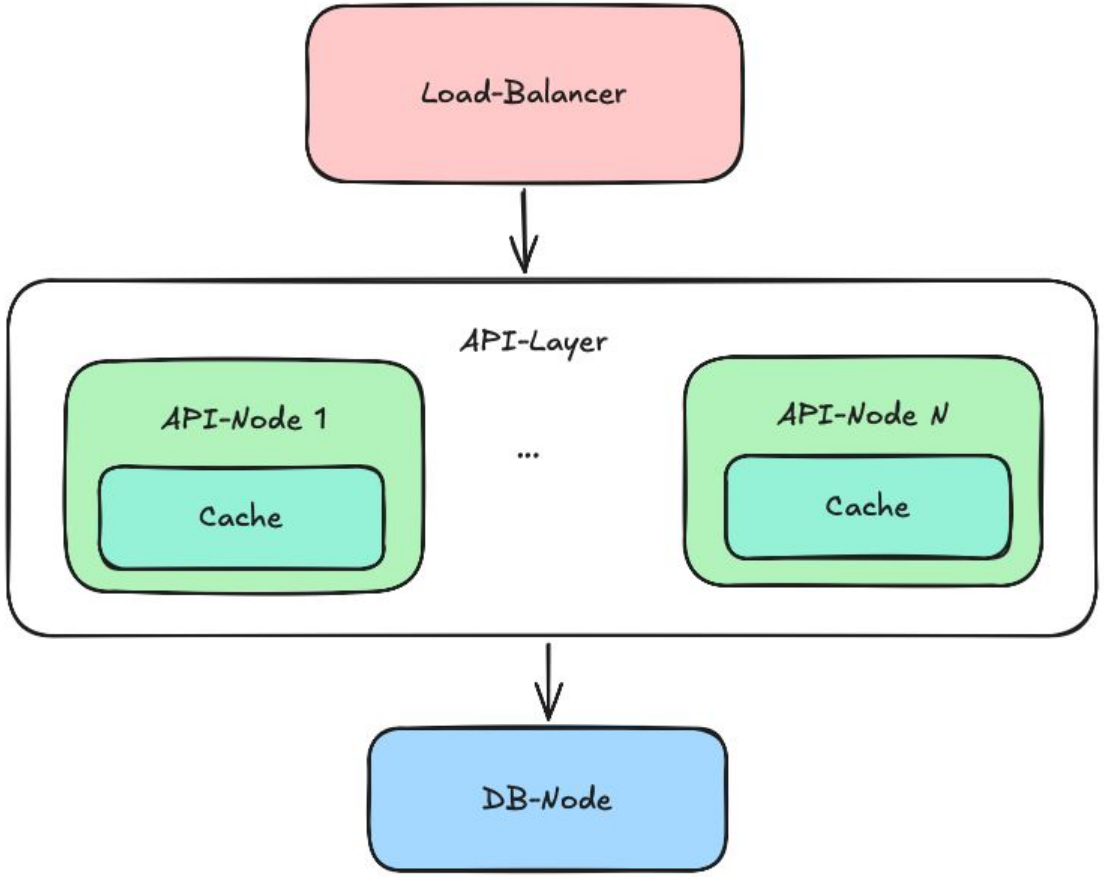
Scalable API-Nodes - Results



- 5 API-Nodes
- Throughput ~1,5k req/s
- DB-Node becomes the bottleneck

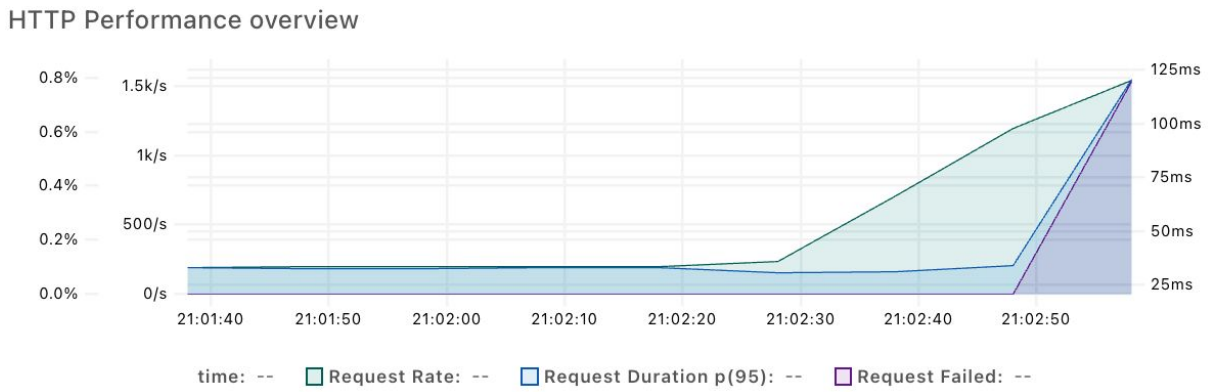
➔ **Add caches to reduce number of requests to DB**

Scaling Strategy 1: Caching



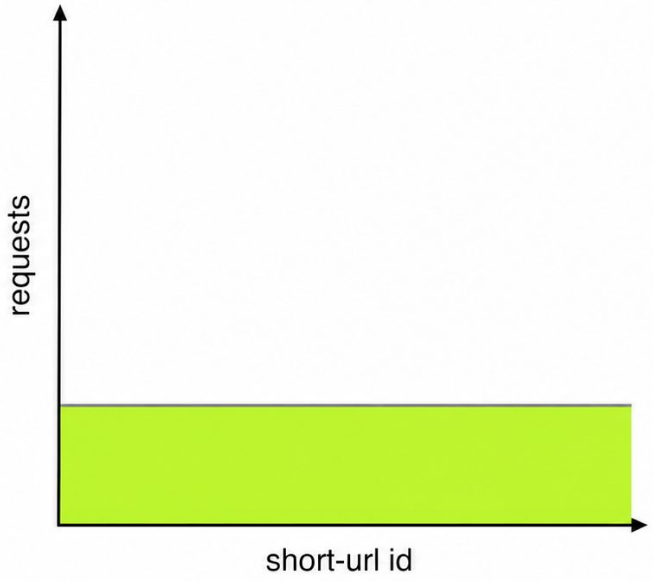
Caching - Results

- 5 API-Nodes
- Throughput ~1,5k req/s
- DB is still the bottleneck

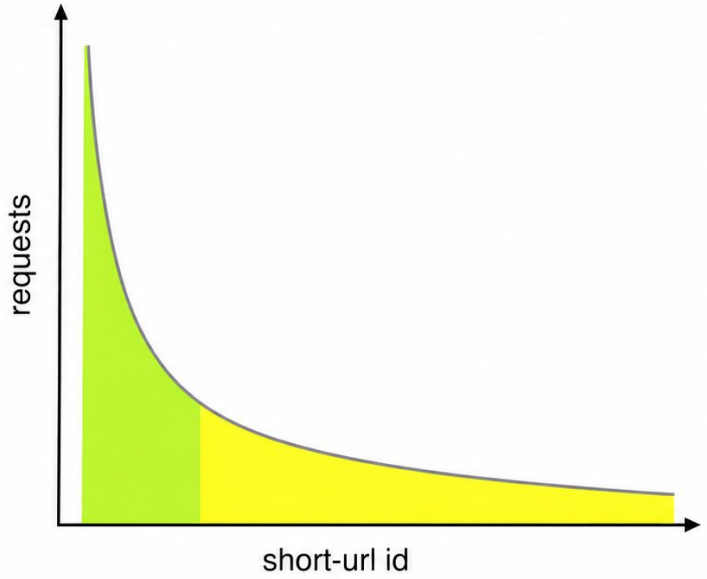


Caching - Results

Uniform Distribution

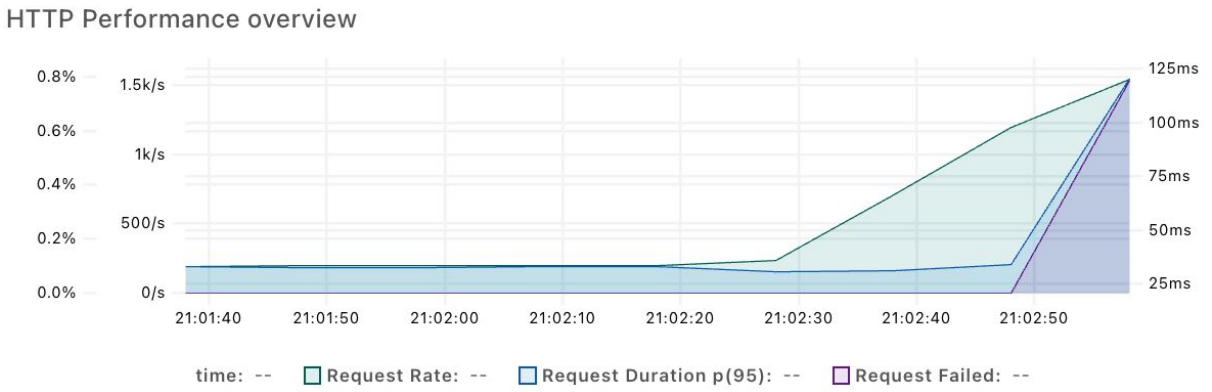


Power law Distribution



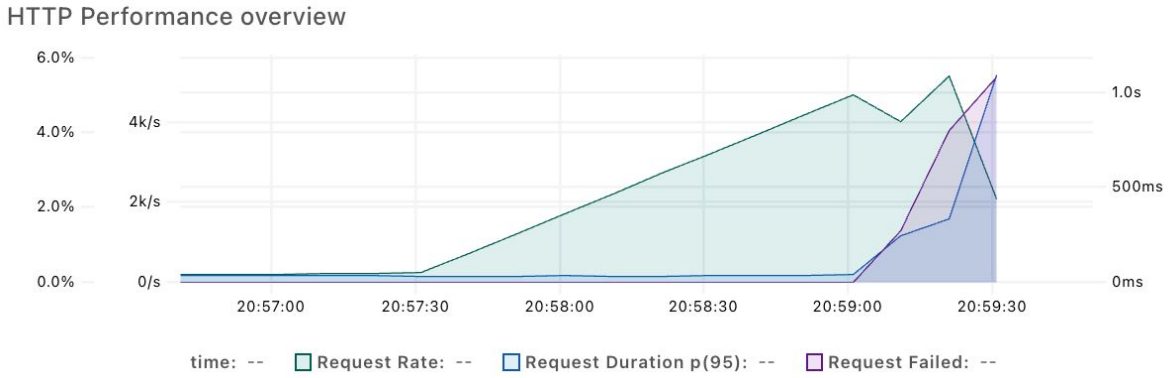
Caching - Results

- 5 API-Nodes, 1 DB-Node
- Read-Only, *Uniform-Distribution*
- Throughput ~1,5k req/s
- DB is still the bottleneck



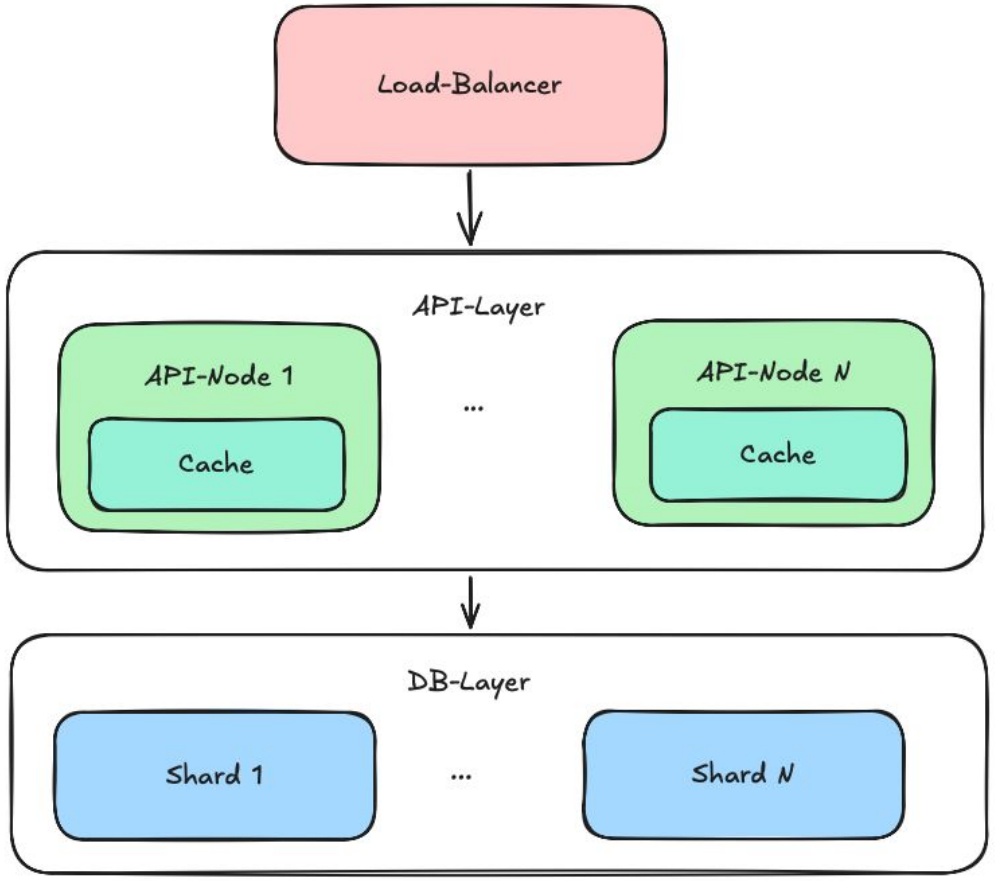
But…

- 5 API-Nodes
- Read-Only, *Hotspot-Distribution*
- Throughput ~5k req/s

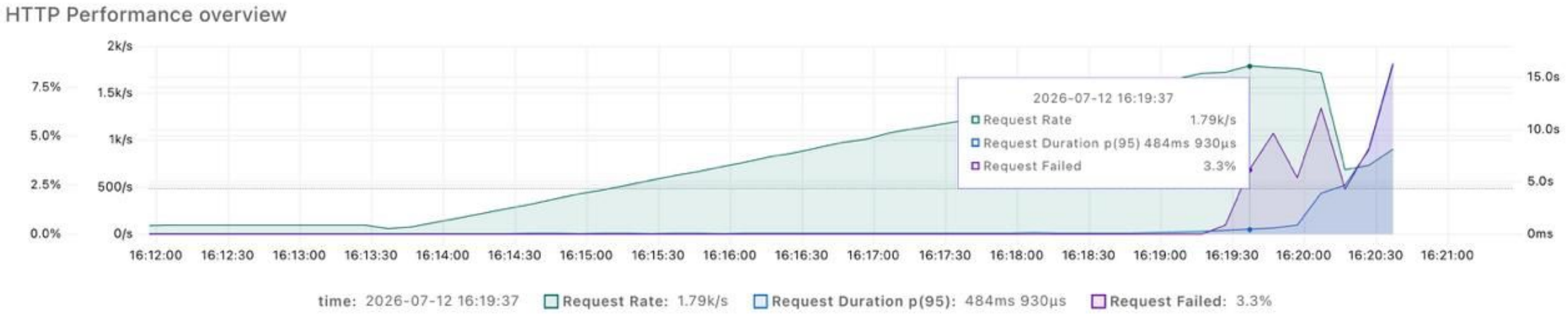


➔ **Caches only help with read-requests and skewed access pattern**

Scaling Strategy 2: Shuffle Sharding



Shuffle Sharding - Results



- 5 API-Nodes, 3 Shards
- Throughput ~1800 req/s

Note: In this Benchmark the DB was hosted in a different region, due to quota limits.

Overload Protection

Ensure that the DB-Layer does not get overloaded when scaling out the API-Layer even further

Overload Protection

Ensure that the DB-Layer does not get overloaded when scaling out the API-Layer even further

Bulkhead (per shard):

➔ Caps concurrent calls a node makes to a shard

➔ If no slot frees up within time, the call is shed

Overload Protection

Ensure that the DB-Layer does not get overloaded when scaling out the API-Layer even further

Bulkhead (per shard):

- ➔ Caps concurrent calls a node makes to a shard
- ➔ If no slot frees up within time, the call is shed

Circuit Breaker (per shard):

- ➔ Opens after consecutive failures to a shard
- ➔ Faster fails
- ➔ Avoids cascading failures
- ➔ Let’s the Node recover without overloading it further

Final Results

Test-scenario:

- 1/3/5 API-Nodes, 3 DB-Nodes
- Read-Only Breakpoint Test, Hotspot Distribution
- Metric Throughput (req/s)

Number of API-Nodes	t2d-standard-1	c4a-standard-1
1	1.03k req/s	1.45k req/s
3	2.96k req/s	4.24k req/s
5	4.94k req/s	5.65k req/s

➔ Near-linear scaling

Note: In this Benchmark the DB was hosted in a different region, due to quota limits.

Possible Scaling Limits

- The API tier cannot be scaled independently forever
- Load-Balancer as single point of failure/potential bottleneck
- Limited Elasticity

Appendix

Sources

Images:

[1] Gargi Ghosal, “How Does a URL Shortener Work?,” MakeUseOf, Jan. 23, 2022. https://www.makeuseof.com/url-shortener-how-does-it-work/ (accessed July 11, 2026).

Tech Stack

Infra / Deployment	Web-Framework	Load Balancing	Load Testing
Terraform, Google Cloud	FastAPI	NGINX	k6, Grafana Labs

Observability	Containerization
Prometheus, Grafana, OpenTelemetry	docker